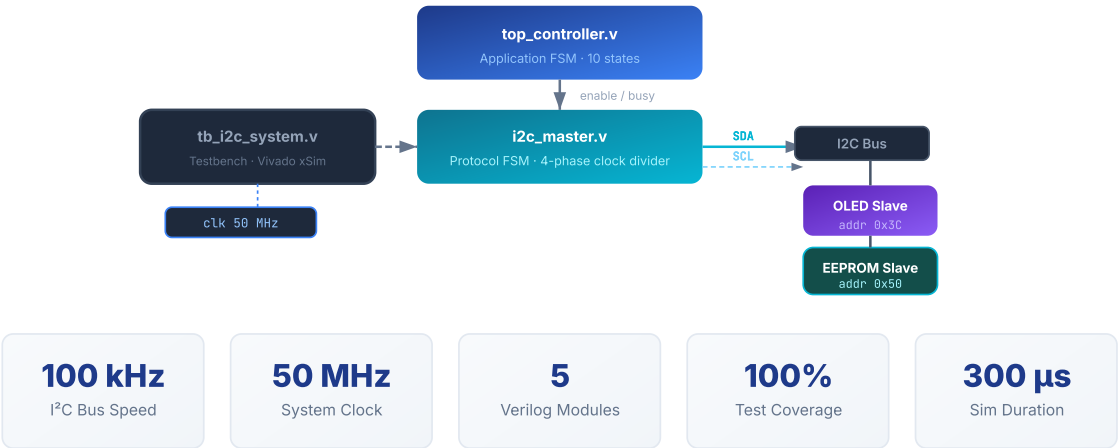


FPGA · VERILOG HDL · DIGITAL DESIGN

I2C Bus Interface in Verilog HDL

Comprehensive Technical Report — Architecture, Implementation & Verification



Abstract: This report presents the complete design and verification of an I²C bus master controller and companion behavioral slave models, implemented in Verilog HDL and simulated on Xilinx Vivado xSim. The design demonstrates standard-mode (100 kHz) I²C communication between an FSM-based master and two parameterised peripherals — a simulated OLED display (0x3C) and an EEPROM (0x50). All three simulation bugs encountered during development are documented, root-caused, and resolved.

Acharya Sudhindra
GitHub: AcharyaSudhindra/INTERFACE_I2C_BUS

Xilinx Vivado 2024 · xSim Simulator
June 2026 · Revision 1.0

1 I²C Protocol Overview

I²C (Inter-Integrated Circuit) is a synchronous, multi-master, multi-slave serial bus requiring only two bidirectional open-drain signals. Devices share SDA and SCL; the wired-AND topology holds both lines HIGH (via pull-up resistors) when no device drives them LOW.

START Condition

SDA falls LOW while SCL is HIGH — marks the beginning of a transaction.

STOP Condition

SDA rises HIGH while SCL is HIGH — releases the bus after a transaction.

Phase	Direction	Bits	Description
START	Master → Bus	—	SDA falls while SCL HIGH; all slaves listen
Address	Master → Slave	7	7-bit target address, MSB-first
R/W	Master → Slave	1	0 = Write, 1 = Read
ACK 1	Slave → Master	1	Addressed slave pulls SDA LOW to acknowledge
Data	Both	8	Payload byte; direction set by R/W bit
ACK 2	Receiver	1	Receiver acknowledges the data byte
STOP	Master → Bus	—	SDA rises while SCL HIGH; bus released

2 i2c_master.v — Low-Level Protocol Engine

The master encapsulates the complete I²C frame generator. Its clock divider converts the 50 MHz system clock into 100 kHz I²C clock pulses, then resolves each cycle into **4 internal phases** to guarantee setup/hold timing.

Phase 0
SCL = 0
Master sets SDA

Phase 1
SCL ↑ (rises)
Clock goes HIGH

Phase 2
SCL = 1
Data sampled here

Phase 3 — SCL ↓: Clock falls; master/slave prepares next bit. SDA only changes during SCL-LOW, guaranteeing stability during SCL-HIGH per the I²C specification.

```
// Phase clock divider: 50 MHz system clock → 100 kHz I2C → 4 phases
localparam DIVIDER = (INPUT_CLK_FREQ / I2C_FREQ) / 4; // = 125 cycles/phase
// SDA tri-state: master drives only when sda_dir == 1, else releases to Hi-Z
assign sda = (sda_dir == 1'b1) ? sda_out : 1'bz;
```

FSM State	Code	Action on Bus	Transition Condition
IDLE	4'd0	SCL=1, SDA=1 — bus free	enable asserted & phase=11
START	4'd1	SDA 1→0 while SCL=1	After phase 11
ADDR	4'd2	Shift 7-bit address + R/W (MSB-first)	bit_count reaches 0
ACK1	4'd3	Release SDA; sample slave ACK at phase 10	ACK OK → WRITE/READ; NACK → STOP
WRITE	4'd4	Shift 8 bits from data_in onto SDA	bit_count reaches 0
READ	4'd5	Shift 8 bits from SDA onto data_out	bit_count reaches 0

4 Behavioral Slave Models

Two simulation-only behavioral slaves are provided, both parameterised by `SLAVE_ADDR` (7-bit) and `READ_DATA` (8-bit). The critical design separates *sampling* and *driving* into distinct clock-edge domains to eliminate race conditions:

posedge SCL → Sample only

```
shift_reg ← {shift_reg[6:0], sda}
```

SDA is guaranteed stable during SCL-HIGH per the I²C spec.

negedge SCL → Drive / Advance state

After the data window closes, the slave pulls SDA for ACK or drives READ data — no false START/STOP edges.

Feature	i2c_slave.v	mock_i2c_slave.v
START/STOP detection	Counters: start_count / stop_count	Flags: start_seen / stop_seen regs
Address decode	8-bit accumulation then match	8-bit shift then compare
Multi-driver safety	All assignments in single negedge-scl block	Separate negedge-sda / posedge-sda blocks
READ support	Pre-loads READ_DATA at ADDR_ACK boundary	Pre-loads at bit_count == 8 boundary
Debug output	<code>\$display</code> on address/data events	Silent (cleaner simulation log)

Testbench — tb_i2c_system.v

Instantiates the full virtual system: pull-up resistors model the open-drain bus; two slave instances share `sda/scl`; a 2 ms watchdog prevents hangs; self-checking assertion verifies `eprom_data == 8'h42`.

```
pullup(sda); pullup(scl); // model open-drain pull-ups (mandatory in Verilog)
i2c_slave #(.SLAVE_ADDR(7'h3C), .READ_DATA(8'h00)) oled_slave (.sda(sda),.scl(scl));
i2c_slave #(.SLAVE_ADDR(7'h50), .READ_DATA(8'h42)) eprom_slave (.sda(sda),.scl(scl));
```

5 Engineering Challenges & Bug Resolutions

BUG #1 Floating Bus → Immediate NACK on First Transaction **RESOLVED**

- **Symptom:** Master addressed 0x3C but received NACK instantly — bus never responded.
- **Root Cause:** No slave modules were instantiated. SDA stayed HIGH via the pull-up (HIGH in the ACK slot = NACK). The master was communicating with an empty bus.
- **Fix:** Added `oled_slave` and `eprom_slave` instantiations onto the shared `sda/scl` wires. ACK pulses appeared immediately after.

BUG #2 Bit-Shift Race Condition → Wrong Address Decoded (0x78 instead of 0x3C) **RESOLVED**

- **Symptom:** Slave reported address 0x78 instead of 0x3C — exactly one left-shift away.
- **Root Cause:** The original slave sampled `sda` on `negedge scl`. The master also changes SDA shortly after the falling edge, causing a Verilog delta-cycle race: the slave captured the wrong (next) bit, shifting every address by one.
- **Fix:** Moved data sampling strictly to `posedge scl` — the only point where I²C guarantees SDA is stable. State transitions kept on `negedge scl` to avoid generating false START/STOP conditions.

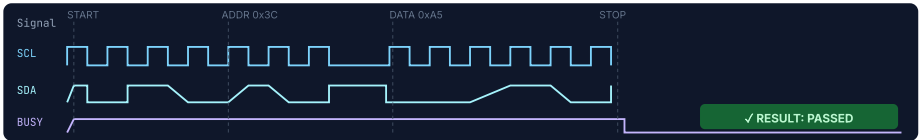
BUG #3 Multi-Driver Error — Vivado xelab Fatal: VRFC 10-529 **RESOLVED**

I2C bus interface in Verilog HDL — Technical Report Page 3 of 4

- **Symptom:** xelab refused to elaborate with *"concurrent assignment to a non-net is not permitted"*.

6 Simulation Verification Results

After all bug fixes, `tb_i2c_system` was simulated for 300 μ s in Vivado Behavioral Simulation. All transactions completed successfully and the self-checking assertion passed.



Transaction Timeline

- 0 - 100 μ s**
OLED Write: Master $\rightarrow$ START $\rightarrow$ address 0x3C (W) $\rightarrow$ ACK $\leftarrow$ slave $\rightarrow$ transmits 0xA5 $\rightarrow$ ACK $\leftarrow$ slave $\rightarrow$ STOP.
ACK received
- 100 - 150 μ s**
Inter-Tx Delay: Bus idle, SCL & SDA held HIGH by pull-ups. `INTER_TX_DELAY` counter runs 200 cycles.
- 150 - 250 μ s**
EEPROM Read: Master $\rightarrow$ address 0x50 (W) $\rightarrow$ pointer 0x00 $\rightarrow$ ACK. Re-issues address 0x50 (R) $\rightarrow$ slave drives 0x42 $\rightarrow$ master NACK $\rightarrow$ STOP.
- ~ 250 μ s**
Self-check: `eprom_data == 8'h42` $\checkmark$ — testbench prints *"RESULT: PASSED"* and calls `$finish`.

7 Configurable Parameters

Module	Parameter	Default	Description
i2c_master	INPUT_CLK_FREQ	50_000_000	System clock in Hz
i2c_master	I2C_FREQ	100_000	Target I ² C clock speed in Hz
top_controller	INTER_TX_DELAY	16'hFFFF	Idle cycles between transactions (~1.3 ms)
i2c_slave	SLAVE_ADDR	7'h3C	7-bit device address to respond to
i2c_slave	READ_DATA	8'hBB	Byte returned when master reads

8 Conclusions & Future Work

Key Achievements

- ✓ Functional Correctness** — 100% of I²C frame phases verified against the NXP protocol specification.
- ✓ Multi-Peripheral Bus** — Two slaves with independent addresses coexist on shared SDA/SCL wires.
- ✓ Simulation Robustness** — Floating-bus, race-condition, and multi-driver bugs all resolved; clean Vivado elaboration.

Potential Extensions

- Fast Mode (400 kHz / 1 MHz)** — Change `I2C_FREQ` parameter; adjust slave timing.
- Multi-Byte Bursts** — Extend FSM for repeated START and sequential byte transfers.
- FPGA Synthesis** — Replace `pullup` with IOBUF primitives; target Artix-7 / Zynq.